CINEBOOK: A MOVIE TICKET BOOKING SYSTEM

DBMS Course Project (CS F212 Database Systems) — Mid-Semester Submission

Group-17, BITS Pilani, Hyderabad Campus | 11 October 2026

Harinandha L (2024A4PS0660H), Imran Hafeez S (2024A4PS0690H), Arjun Rathore (2024A4PS0691H),
Sushanth Reddy Kompally (2024A8PS0459H), Pratham Singla (2024A8PS0486H)

1 Problem Statement and Scope

Online ticketing platforms must let many users browse shows and reserve seats at the same time without ever selling the same seat twice. CineBook is a simplified BookMyShow-style system covering cities, venues, screens, movies, shows, seat inventory, bookings, payments and reporting. The project concentrates on relational design and DBMS-level integrity; the application layer exists to demonstrate them. **In scope:** show search and filtering, seat selection with a temporary hold, simulated payment, cancellation with refund, show scheduling and admin analytics. **Out of scope:** real payment gateways, food ordering, dynamic pricing and a seat-map editor.

Table 1: Intended users and main functionality.

User	Main functionality
Customer	Register/login; search shows by city, movie, language, date and price; pick seats; pay; view booking history; cancel and receive a refund.
Venue manager	Schedule shows on the screens of own venues; set per-category seat prices; view own occupancy and revenue.
Admin	Maintain cities, venues, screens (with seat layouts) and movies; view dashboards (revenue per venue, occupancy, top movies per city).

2 Assumptions and Business Rules

- **R1** A seat is identified only within its screen (weak entity) and has one category: REGULAR, PREMIUM or RECLINER.
- **R2** A seat has at most one *active* booking per show; cancelled or expired bookings free the seat but stay as history.
- **R3** A show is of one movie on one screen; two shows on the same screen may not overlap in time.
- **R4** The listed price is the SHOW_PRICE of the show and the seat's category. It is copied into the reservation as **list_price** (a snapshot). **price_paid** (list price minus the seat's share of the discount) exists only after confirmation and is NULL until then.
- **R5** A booking is for one show and has 1 to 6 seats. It starts HELD with a 10-minute expiry (the amount payable is already fixed), becomes CONFIRMED only after a successful payment, and otherwise ends CANCELLED or EXPIRED. Expired holds are released at the start of every booking attempt and by a scheduled clean-up.
- **R6** Cancellation is allowed until 2 hours before the show. Assumed refund: 100% if more than 24 h before, 50% between 2 h and 24 h, computed from price_paid and recorded in PAYMENT.refund_amount.
- **R7** A payment belongs to one booking; a retry creates a new payment record. Payments are simulated.
- **R8** Every venue has exactly one manager (a user with role MANAGER); every user has exactly one role. **R9** Every booking status change is logged (BOOKING_AUDIT).
- **R10** A booking may use one coupon (percentage or flat, with expiry and optional usage limit). discount_amount is computed when the coupon is applied: total_amount (amount payable) = sum of list_price − discount_amount, and a coupon never makes a booking free. At confirmation the discount is spread over the seats so that the sum of price_paid equals total_amount.
- **R11** For fast ticket display, a booking stores a write-once snapshot of the show details (movie title, venue, screen, start time), like an invoice.

3 ER/EER Design

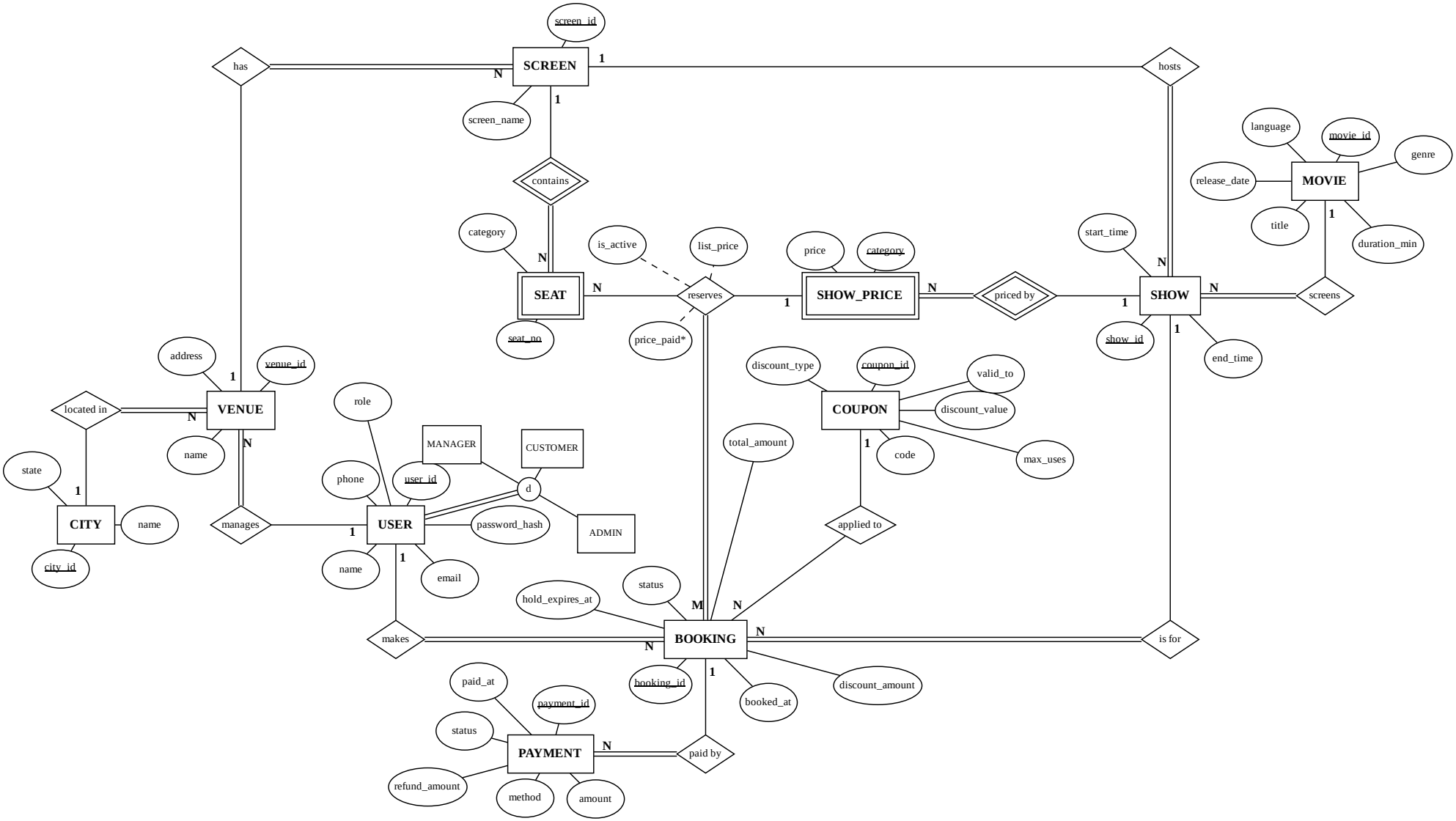


Figure 1: Conceptual ER/EER diagram of CineBook (Chen notation).

Table 2: Relationships, cardinality and participation (total = every instance must take part).

Relationship	Entities	Card.	Participation
located in	CITY, VENUE	1:N	VENUE total
has	VENUE, SCREEN	1:N	SCREEN total
contains (identifying)	SCREEN, SEAT	1:N	SEAT total (weak, partial key seat_no)
hosts / screens	SCREEN, MOVIE, SHOW	1:N each	SHOW total in both
priced by (identifying)	SHOW, SHOW_PRICE	1:N	SHOW_PRICE total (weak, partial key category)
makes / is for	USER, SHOW, BOOKING	1:N each	BOOKING total in both
paid by	BOOKING, PAYMENT	1:N	PAYMENT total
applied to	COUPON, BOOKING	1:N	both partial (coupon optional)
manages	USER (MANAGER), VENUE	1:N	VENUE total (maps to VENUE.manager_id)
reserves (ternary)	BOOKING, SEAT, SHOW_PRICE	M:N:1	BOOKING total. Each reservation uses exactly one SHOW_PRICE (the booking's show, the seat's category); attributes list_price, price_paid, is_active; maps to BOOKING_SEAT

EER notes. USER is specialised into CUSTOMER, MANAGER and ADMIN (disjoint, total); the subclasses have no extra attributes, so the specialisation is mapped to a single relation with a type attribute *role* (mapping option 8C). SEAT and SHOW_PRICE are weak entities. The ternary *reserves* becomes the relation BOOKING_SEAT, whose foreign keys to BOOKING, SEAT and SHOW_PRICE enforce the rules declaratively. BOOKING additionally stores write-once sale-time snapshot columns (movie_title, venue_name, screen_name, show_start), and BOOKING_SEAT stores seat_category; these are added during mapping and are not drawn (Section 6).

4 Relational Schema (13 Relations)

Notation: underlined = primary key; (FK) = foreign key; NN = NOT NULL. Timestamps are TIMESTAMPTZ; surrogate keys are integer identity columns.

Table 3: Relational schema with keys and constraints.

Relation	Attributes	Keys and constraints
APP_USER	(user_id, name, email, phone, password_hash, role)	email UNIQUE NN; role CHECK IN (CUSTOMER, MANAGER, ADMIN)
CITY	(city_id, name, state)	UNIQUE (name, state)
VENUE	(venue_id, name, address, city_id, manager_id)	city_id → CITY; manager_id → APP_USER (trigger: role = MANAGER); all NN
SCREEN	(screen_id, venue_id, screen_name)	venue_id → VENUE ON DELETE CASCADE; UNIQUE (venue_id, screen_name)
SEAT	(screen_id, seat_no, category)	screen_id → SCREEN CASCADE; category CHECK IN (REGULAR, PREMIUM, RECLINER); UNIQUE (screen_id, seat_no, category)
MOVIE	(movie_id, title, duration_min, language, genre, release_date)	title, language NN; duration_min CHECK > 0
MOVIE_SHOW	(show_id, movie_id, screen_id, start_time, end_time)	movie_id → MOVIE; screen_id → SCREEN; CHECK end $>$ start; UNIQUE (show_id, screen_id); EXCLUDE (screen_id =, time range &&) forbids overlap
SHOW_PRICE	(show_id, category, price)	show_id → MOVIE_SHOW CASCADE; category CHECK as SEAT; price CHECK > 0
COUPON	(coupon_id, code, discount_type, discount_value, valid_to, max_uses)	code UNIQUE NN; type CHECK IN (PERCENT, FLAT); value CHECK > 0 (≤ 100 for PERCENT); max_uses NULL = unlimited
BOOKING	(booking_id, user_id, show_id, coupon_id, status, booked_at, hold_expires_at, discount_amount, total_amount, *movie_title, venue_name, screen_name, show_start*)	user_id → APP_USER; show_id → MOVIE_SHOW; coupon_id → COUPON (NULL = none); status CHECK IN (HELD, CONFIRMED, CANCELLED, EXPIRED); discount_amount ≥ 0; total_amount > 0; snapshot columns (italic) NN, written once; UNIQUE (booking_id, show_id); index (user_id, booked_at DESC)
BOOKING_SEAT (from *reserves*)	(booking_id, screen_id, seat_no, show_id, *seat_category, list_price*, price_paid, is_active)	FKs: (booking_id, show_id) → BOOKING; (show_id, screen_id) → MOVIE_SHOW; (screen_id, seat_no, seat_category) → SEAT; (show_id, seat_category) → SHOW_PRICE; list_price > 0; **price_paid NULL until confirmed, then 0 $\leq$ price_paid $\leq$ list_price**; **partial UNIQUE index (show_id, screen_id, seat_no) WHERE is_active**
PAYMENT	(payment_id, booking_id, method, amount, status, paid_at, refund_amount)	booking_id → BOOKING NN; method CHECK IN (UPI, CARD, NETBANKING, WALLET); status CHECK IN (INITIATED, SUCCESS, FAILED, REFUNDED); amount > 0; refund_amount NULL or $0 \leq$ refund $\leq$ amount
BOOKING_AUDIT	(audit_id, booking_id, old_status, new_status, changed_at)	booking_id → BOOKING; filled only by a trigger

Why four composite foreign keys in BOOKING_SEAT? A seat can only be booked if (a) the booking belongs to the same show, (b) the show runs on the seat's screen, (c) the seat exists with the stored category, and (d) that category has a

SHOW_PRICE row for the show. The DBMS rejects any inconsistent row without a trigger. Together with the partial unique index and the exclusion constraint, the main rules live in the schema itself.

5 Integrity Constraints Enforced by the DBMS

- **Keys and referential integrity:** a primary key on every relation and a foreign key for every relationship; CASCADE only for owned weak entities (SEAT, SHOW_PRICE, SCREEN under VENUE), RESTRICT for history (BOOKING, PAYMENT, BOOKING_AUDIT).
- **Domain constraints:** NOT NULL on mandatory attributes; CHECK on role, category, statuses, payment method, prices (> 0), durations, amounts, time order and $0 \leq$ price_paid $\leq$ list_price.
- **No double booking (R2):** the partial unique index allows one active row per (show, seat); of two concurrent transactions for the same seat the second fails with a unique violation.
- **No overlapping shows (R3):** an exclusion constraint on MOVIE_SHOW (same screen, overlapping time range); back-to-back shows are allowed (half-open range).
- **Rules a CHECK cannot express (triggers/procedures):** 1 to 6 seats per booking; manager must have role MANAGER; every status change writes BOOKING_AUDIT; coupon validity, usage limit and discount are computed in the booking procedure, which locks the coupon row to stay race-safe. *Confirmation rule* (deferred constraint trigger): a CONFIRMED booking has a SUCCESS payment equal to total_amount, every active seat has price_paid set, and sum(price_paid) = total_amount. A trigger sets is_active = false when a booking becomes CANCELLED or EXPIRED.

6 Normalization and Design Justification

Table 4: Functional dependencies and normal forms.

Relation	Non-trivial functional dependencies	Normal form
APP_USER, CITY, VENUE, SCREEN, MOVIE, COUPON, PAYMENT, BOOKING_AUDIT	Every non-key attribute depends on the surrogate key; other candidate keys: email, code, (name, state), (venue_id, screen_name)	BCNF
SEAT, SHOW_PRICE, MOVIE_SHOW	(screen_id, seat_no) $\rightarrow$ category; (show_id, category) $\rightarrow$ price; show_id $\rightarrow$ movie_id, screen_id, times; (screen_id, start_time) $\rightarrow$ show_id	BCNF
BOOKING	booking_id $\rightarrow$ all; show_id $\rightarrow$ movie_title, venue_name, screen_name, show_start (snapshot)	Not 3NF, by design
BOOKING_SEAT	key $\rightarrow$ rest; booking_id $\rightarrow$ show_id; show_id $\rightarrow$ screen_id; (screen_id, seat_no) $\rightarrow$ seat_category; list_price is a snapshot, so (show_id, seat_category) $\rightarrow$ list_price is not enforced	Not 2NF, by design

Deliberate denormalizations (each documented and controlled):

- **Sale-time snapshots** (BOOKING.movie_title, venue_name, screen_name, show_start; and BOOKING_SEAT.seat_category, list_price): like an invoice, a ticket must show what was sold even if a movie is renamed or a price changes. They are written once and never updated, so there is no update anomaly.
- **BOOKING_SEAT.show_id, screen_id** repeat what booking and show determine; they let the partial unique index and composite FKs enforce "one active booking per seat per show" in the DBMS, and the FKs make drift impossible. **is_active** mirrors the booking status because a unique index cannot look into another table; a trigger keeps them in step.
- **total_amount** = sum(list_price) $-$ discount_amount is derived and maintained only by the booking procedure.

Read-path design (API performance). A naive ticket page joins APP_USER, BOOKING, BOOKING_SEAT and SEAT, plus MOVIE_SHOW, MOVIE, SCREEN and VENUE. Here, BOOKING_SEAT is a self-contained ticket line (seat_no is the label; category and prices are stored), BOOKING holds the display snapshot, and the user comes from the JWT. *Ticket detail* is one primary-key lookup on BOOKING plus a range scan of at most 6 BOOKING_SEAT rows; *my bookings* is one scan of the index (user_id, booked_at DESC) with no joins. Joins stay in the admin reports. Further options for real load: caching confirmed tickets, read replicas and partitioning BOOKING by date. The final submission will compare EXPLAIN ANALYZE of the naive join and the snapshot query on seeded data.

7 Application Design

Architecture: Front End $\leftrightarrow$ Application/Backend $\leftrightarrow$ Relational Database. Business-critical rules live in the database; the backend orchestrates transactions and the front end presents data.

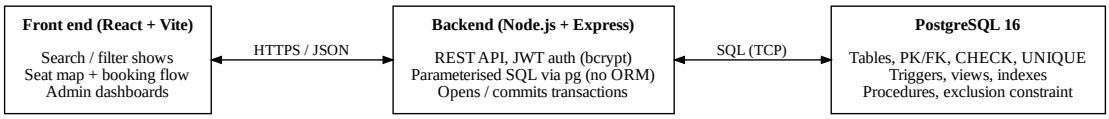


Figure 2: Proposed three-tier architecture.

Table 5: Proposed technology stack.

Layer	Technology
Front end	React with Vite; Axios; role-based routes for customer, manager and admin
Backend	Node.js with Express; REST API; JWT authentication; bcrypt; node-postgres with parameterised SQL and explicit transactions (no ORM)
Database	PostgreSQL 16 (btree_gist extension for the exclusion constraint)
Tooling	Docker Compose; Python Faker script for 500+ records; Git/GitHub

Table 6: Planned end-to-end workflows.

#	Workflow	What it demonstrates
W1	Book tickets: search, choose seats, hold (list_price copied from SHOW_PRICE), apply coupon, pay, confirm (price_paid set); cancellation with refund	*Transaction* over BOOKING, BOOKING_SEAT, PAYMENT; rollback on payment failure; seat races resolved by the unique index
W2	Admin dashboard: revenue per venue per month, occupancy per show, top 3 movies per city, peak hours	*Complex retrieval:* joins, GROUP BY, subqueries, CTEs, views
W3	Schedule shows; attempt an overlapping show, double booking, 7-seat booking, bad price, expired coupon	*Constraint enforcement:* exclusion constraint, partial unique index, CHECK, triggers

Planned DBMS features: transactions (W1), triggers (audit log, seat-count rule, manager check), views (availability, revenue), indexes with an EXPLAIN ANALYZE comparison, and a stored procedure for cancellation and refund; at least five non-trivial queries come from W2. Hot read paths `GET /bookings/:id` and `GET /me/bookings` touch only BOOKING and BOOKING_SEAT.

8 Progress and Team Contributions

Requirements, ER/EER model, relational mapping, constraints, normalization and application design are **done**; DDL scripts, triggers, procedures, seed data, backend and front end are **planned** for completion before the 15 November final submission.

Table 7: Contributions up to mid-semester.

Member	ID	Contribution
Pratham Singla	2024A8PS0486H	Core database design: ER/EER model, relational mapping, keys and integrity constraints, normalization, design justification, booking transaction and concurrency design
Harinandha L	2024A4PS0660H	Requirements analysis, users, assumptions and business rules
Imran Hafeez S	2024A4PS0690H	Application design: workflows, technology stack, architecture
Arjun Rathore	2024A4PS0691H	GitHub repository setup, README, data-population plan
Sushanth Reddy Kompally	2024A8PS0459H	Planned SQL queries, views and reporting design for the dashboard

GitHub repository: `https://github.com/Pratham15-svg/CineBook`